

TUGAS SOFTWARE QUALITY ASSURANCE

Nama : Yoga Panggabean

NIM : 2310031802038

Hari, Tanggal : Kamis, 21 Mei 2026

Soal

Pilih salah satu dari ketiga tools di bawah ini:

- a) Jira Software, paling populer untuk manajemen defect yang terintegrasi dengan metode Agile.
- b) GitHub Issues, sangat cocok bagi pengembang karena menyatu langsung dengan repositori kode.
- c) Bugzilla, tools pelacakan klasik yang gratis, tangguh, dan sangat spesifik untuk bug tracking

Jawab

1. Alasan Memilih GitHub Issues

Tools yang dipilih dalam laporan ini adalah GitHub Issues. Dibandingkan Jira Software dan Bugzilla, GitHub Issues dipilih karena paling sederhana untuk dipelajari oleh mahasiswa dan pengembang pemula. GitHub Issues sudah tersedia di dalam repository GitHub, sehingga pengguna tidak perlu melakukan instalasi server atau konfigurasi yang rumit. GitHub sendiri menjelaskan bahwa GitHub Issues dibangun langsung di setiap repository dan dapat digunakan untuk merencanakan kerja, melacak masalah, serta berdiskusi terkait software (GitHub, 2026).



Gambar 1. Ilustrasi GitHub Issues sebagai tool pelacakan bug

GitHub Docs juga menjelaskan bahwa issues dapat digunakan untuk melacak ide, feedback, tugas, atau bug. Artinya, satu bug dapat dicatat sebagai issue, diberi label, ditugaskan kepada anggota tim, ditautkan dengan pull request, lalu ditutup ketika perbaikan selesai (GitHub Docs, 2026a). Karena alasan tersebut, GitHub Issues menjadi pilihan yang paling mudah, cepat, dan cocok untuk laporan ini.

Tabel 1. Pertimbangan Pemilihan Tools

Tools	Kelebihan Utama	Alasan Tidak Dipilih / Dipilih
Jira Software	Fitur Agile dan defect management sangat lengkap.	Lebih kompleks, banyak konfigurasi, dan laporan bisa terlalu panjang untuk tugas sederhana.
GitHub Issues	Langsung menyatu dengan repository kode, mudah dibuat, mudah diberi label, dan cocok untuk developer.	Dipilih karena paling mudah digunakan dan sumber referensinya banyak.
Bugzilla	Gratis, klasik, dan fokus pada bug tracking.	Tampilannya lebih teknis dan kurang familiar bagi pemula dibanding GitHub Issues.

2. Pengertian GitHub Issues

GitHub Issues adalah fitur GitHub yang digunakan untuk mencatat, mendiskusikan, mengelola, dan menelusuri pekerjaan dalam sebuah repository. Dalam konteks Software Quality Assurance, GitHub Issues dapat dipakai sebagai alat pelacakan bug. Bug yang ditemukan oleh tester atau pengguna dapat dibuat menjadi issue baru, kemudian diberi informasi seperti judul bug, deskripsi, langkah reproduksi, hasil yang diharapkan, hasil aktual, label, prioritas, assignee, milestone, dan lampiran bukti.

Kelebihan utama GitHub Issues adalah integrasinya dengan aktivitas pengembangan perangkat lunak. Sebuah issue dapat dikaitkan dengan pull request, commit, branch, dan project board. Jika developer memperbaiki bug melalui pull request dan menggunakan kata kunci seperti fixes #10, maka issue yang terkait dapat tertutup otomatis ketika pull request tersebut digabung ke default branch (GitHub Docs, 2026c).



Gambar 2. Contoh tampilan daftar bug dalam GitHub Issues

3. Sejarah dan Perkembangan Singkat

GitHub dikenal sebagai platform untuk menyimpan dan mengelola kode menggunakan Git. Seiring berkembangnya kebutuhan kolaborasi *software*, GitHub tidak hanya digunakan untuk menyimpan kode, tetapi juga untuk diskusi, pull request, code review, automation, project planning, dan issue tracking. GitHub Issues menjadi salah satu fitur penting karena membuat pekerjaan pengembangan berada dekat dengan kode.

Pada perkembangan terbaru, GitHub Issues semakin terhubung dengan GitHub Projects. GitHub Projects menyediakan tampilan tabel, board, dan roadmap yang dapat digunakan untuk merencanakan serta melacak pekerjaan dari issue dan pull request. Integrasi ini membuat tim dapat melihat backlog, sprint, status pekerjaan, dan bug dalam satu tempat (GitHub Docs, 2026e).

Tabel 2. Garis Waktu Ringkas Perkembangan GitHub Issues

Periode	Perkembangan	Makna untuk Bug Tracking
2008	GitHub mulai dikenal sebagai platform kolaborasi kode berbasis Git.	Kode dan diskusi pengembangan mulai berada dalam satu platform.
2010-an	Fitur issues digunakan luas oleh proyek open source.	Bug report, pertanyaan, dan request fitur dapat dikelola dalam repository.
2016+	Issue templates makin banyak digunakan untuk membantu pelaporan bug yang lebih rapi.	Reporter bug diarahkan mengisi data penting seperti langkah reproduksi dan environment.
2020-an	GitHub Projects berkembang dengan view tabel, board, roadmap, custom fields, dan insights.	Bug dapat dipantau seperti item kerja dalam backlog atau board QA.

Saat ini	Issues, pull request, branch, label, milestone, project, dan automation saling terhubung.	Pelacakan bug menjadi lebih sederhana dan dekat dengan proses perbaikan kode.
----------	---	---

4. Konsep Dasar dan Komponen Utama

Agar GitHub Issues dapat digunakan sebagai tools bug tracking, pengguna perlu memahami beberapa komponen utama. Komponen ini tidak serumit Jira, tetapi sudah cukup untuk kebutuhan pengelolaan defect pada proyek kecil sampai menengah. Komponen seperti issue, label, assignee, milestone, pull request, dan project board membantu tim mengetahui bug apa yang masih terbuka, siapa yang bertanggung jawab, serta bagaimana status perbaikannya.

Tabel 3. Komponen Inti GitHub Issues dan Perannya

Komponen	Pengertian	Peran dalam Bug Tracking
Repository	Tempat menyimpan kode, dokumentasi, dan riwayat perubahan.	Bug dicatat dekat dengan kode yang bermasalah.
Issue	Catatan pekerjaan atau masalah yang perlu dibahas dan diselesaikan.	Satu bug sebaiknya dibuat menjadi satu issue agar mudah dilacak.
Label	Penanda kategori seperti bug, enhancement, priority, atau area modul.	Membedakan bug dari tugas lain dan memudahkan filter.
Assignee	Orang yang ditugaskan menangani issue.	Menjelaskan siapa yang bertanggung jawab memperbaiki bug.
Milestone	Target versi, sprint, atau batas waktu tertentu.	Mengelompokkan bug yang harus selesai sebelum rilis.
Issue type	Jenis issue seperti task, bug, atau feature.	Membantu klasifikasi masalah sesuai jenis pekerjaan.
Pull Request	Usulan perubahan kode yang akan direview dan digabung.	Perbaikan bug dapat ditautkan langsung ke issue.
Projects	Board/tabel/roadmap untuk merencanakan dan memantau pekerjaan.	Bug dapat dilihat dalam kolom backlog, progress, review, dan done.

5. Cara Kerja GitHub Issues untuk Manajemen Bug

Secara umum, GitHub Issues bekerja dengan menjadikan bug sebagai item kerja yang dapat dilacak dari awal hingga selesai. Ketika bug ditemukan, reporter membuat issue dengan

informasi yang jelas. Setelah itu, maintainer atau QA melakukan triage untuk memastikan apakah bug valid, duplikat, membutuhkan informasi tambahan, atau perlu diprioritaskan.

Jika bug dinyatakan valid, issue diberi label, assignee, milestone, dan dapat dimasukkan ke project board. Developer kemudian memperbaiki bug melalui branch atau pull request. Setelah perbaikan selesai, tim melakukan review dan pengujian ulang. Issue dapat ditutup jika bug sudah benar-benar diperbaiki.



Gambar 3. Alur sederhana pelacakan bug menggunakan GitHub Issues

5.1 Alur End-to-End Bug dalam GitHub Issues

- a. **Identifikasi bug.** Tester, user, atau developer menemukan masalah pada aplikasi. Contohnya, tombol checkout tidak merespon di perangkat mobile.
- b. **Membuat issue baru.** Bug dilaporkan melalui tab Issues dengan judul yang jelas, deskripsi, langkah reproduksi, hasil yang diharapkan, hasil aktual, environment, dan bukti berupa screenshot atau video.
- c. **Triage.** Maintainer atau QA lead memeriksa apakah bug valid, duplikat, perlu informasi tambahan, atau sebenarnya permintaan fitur.
- d. **Pemberian metadata.** Issue diberi label seperti bug, P1, frontend, backend, atau payment. Issue juga dapat diberi assignee, milestone, dan project board.
- e. **Perbaiki kode.** Developer membuat branch atau pull request. Pull request dapat ditautkan ke issue secara manual atau menggunakan kata kunci seperti fixes #10 (GitHub Docs, 2026c).
- f. **Review dan testing ulang.** Tim melakukan code review dan QA melakukan retest. Jika bug masih muncul, issue dapat dibuka kembali atau dikembalikan ke developer.

- g. **Penutupan issue.** Issue ditutup jika perbaikan sudah benar, sudah digabung, dan sudah sesuai dengan definisi selesai.

6. Fungsi GitHub Issues

Fungsi GitHub Issues tidak hanya untuk mencatat bug. Fitur ini juga dapat digunakan untuk koordinasi pekerjaan, komunikasi tim, pelacakan progres, dan integrasi dengan proses pengembangan kode. GitHub Docs menjelaskan bahwa issue dapat diberi metadata seperti issue type, labels, dan milestones, serta dapat dihubungkan dengan Projects untuk perencanaan kerja (GitHub Docs, 2026a).

Tabel 4. Fungsi GitHub Issues dan Manfaatnya

Fungsi	Deskripsi	Manfaat
Bug tracking	Mencatat bug sebagai issue lengkap dengan komentar dan lampiran.	Bug tidak hilang dalam chat dan dapat ditelusuri.
Triage	Memilah bug valid, duplikat, prioritas tinggi, atau perlu info tambahan.	Tim dapat fokus pada masalah yang benar-benar penting.
Labeling	Memberi label seperti bug, P0, frontend, backend, atau needs info.	Memudahkan pencarian dan pengelompokan bug.
Assignment	Menugaskan issue kepada developer atau anggota tim.	Tanggung jawab menjadi jelas.
Milestone	Mengelompokkan issue untuk rilis, sprint, atau target waktu.	Tim tahu bug mana yang harus selesai sebelum rilis.
Search dan filter	Mencari issue berdasarkan label, assignee, status, type, dan field.	Bug dapat ditemukan dengan cepat.
Pull request link	Menghubungkan issue dengan PR atau branch.	Perbaikan bug bisa ditelusuri sampai kode.
Projects	Menampilkan issue dalam tabel, board, dan roadmap.	Progres bug terlihat seperti papan kerja.



Gambar 4. Contoh project board untuk mengelola bug

7. Kelebihan GitHub Issues

GitHub Issues memiliki beberapa kelebihan sebagai tool pelacakan bug dalam proses Software Quality Assurance. Kelebihan utama dari GitHub Issues adalah kemudahan penggunaan, integrasi langsung dengan repositori kode, serta kemampuannya dalam mendukung kolaborasi antara tester, developer, dan anggota tim lainnya.

a. Mudah digunakan dan dipelajari

GitHub Issues memiliki tampilan yang sederhana dan mudah dipahami. Pengguna cukup membuka repositori, memilih tab Issues, kemudian membuat laporan bug baru. Hal ini membuat GitHub Issues cocok digunakan oleh mahasiswa, pengembang pemula, maupun tim kecil yang membutuhkan tool pelacakan bug tanpa konfigurasi yang rumit.

b. Terintegrasi langsung dengan repositori kode

Salah satu kelebihan utama GitHub Issues adalah integrasinya dengan repositori GitHub. Bug yang dilaporkan dapat dihubungkan langsung dengan branch, commit, dan pull request. Dengan demikian, proses pelacakan bug menjadi lebih jelas karena tim dapat menelusuri hubungan antara laporan masalah dan perubahan kode yang dilakukan untuk memperbaikinya.

c. Mendukung kolaborasi tim

GitHub Issues memungkinkan anggota tim untuk berdiskusi langsung di dalam issue melalui komentar, mention, reaksi, dan lampiran. Tester dapat menjelaskan bug yang ditemukan, developer dapat menanyakan detail tambahan, dan QA dapat memberikan hasil retest pada issue yang sama. Hal ini membuat komunikasi menjadi lebih terpusat dan terdokumentasi.

d. **Memiliki fitur label, assignee, dan milestone**

GitHub Issues menyediakan fitur label untuk mengelompokkan bug berdasarkan kategori, prioritas, atau area modul. Fitur assignee digunakan untuk menentukan siapa yang bertanggung jawab menangani bug, sedangkan milestone dapat digunakan untuk mengelompokkan bug berdasarkan target rilis atau sprint. Ketiga fitur ini membantu proses defect management menjadi lebih terstruktur.

e. **Dapat dihubungkan dengan pull request**

Perbaikan bug dapat ditautkan langsung dengan pull request. Misalnya, ketika developer membuat pull request untuk memperbaiki suatu bug, pull request tersebut dapat dikaitkan dengan issue terkait. Hal ini memudahkan tim dalam memastikan bahwa setiap laporan bug memiliki tindak lanjut yang jelas dalam bentuk perubahan kode.

f. **Mendukung pencarian dan filter issue**

GitHub Issues memiliki fitur pencarian dan filter yang dapat digunakan untuk menemukan bug berdasarkan status, label, assignee, milestone, atau kata kunci tertentu. Dengan fitur ini, tim dapat dengan mudah melihat bug yang masih terbuka, bug prioritas tinggi, bug yang belum memiliki penanggung jawab, atau bug yang sudah selesai diperbaiki.

g. **Dapat digunakan bersama GitHub Projects**

GitHub Issues dapat diintegrasikan dengan GitHub Projects untuk menampilkan daftar bug dalam bentuk board, tabel, atau roadmap. Dengan fitur ini, tim dapat memantau status bug seperti Backlog, To Do, In Progress, Review, dan Done. Hal ini membantu proses pemantauan pekerjaan menjadi lebih visual dan mudah dipahami.

h. **Cocok untuk proyek kecil hingga menengah**

Untuk proyek sederhana, tugas kuliah, proyek open source, atau tim pengembang kecil, GitHub Issues sudah cukup memadai sebagai alat pelacakan bug. Fitur dasarnya sudah dapat mendukung pencatatan bug, pembagian tugas, pemantauan progres, dan dokumentasi perbaikan.

i. **Gratis untuk penggunaan dasar**

GitHub Issues dapat digunakan tanpa biaya tambahan pada repositori GitHub. Hal ini menjadi kelebihan penting, terutama untuk mahasiswa, proyek pembelajaran, maupun tim kecil yang membutuhkan tool defect management dengan biaya rendah

8. Kekurangan dan Risiko Penggunaan

Meskipun GitHub Issues memiliki banyak kelebihan sebagai tool pelacakan bug, terdapat beberapa kekurangan dan risiko yang perlu diperhatikan. Kekurangan tersebut terutama berkaitan dengan keterbatasan fitur defect management tingkat lanjut, pelaporan metrik QA, serta ketergantungan pada kedisiplinan tim dalam mengelola issue.

a. Workflow defect management masih terbatas

GitHub Issues belum menyediakan alur kerja defect management yang sangat kompleks seperti Jira. Pada proyek besar, proses QA biasanya membutuhkan workflow khusus, approval bertingkat, status defect yang lebih detail, serta aturan transisi antarstatus. Fitur tersebut belum tersedia secara lengkap bawaan di GitHub Issues.

b. Severity dan priority tidak tersedia secara otomatis

GitHub Issues tidak secara langsung membedakan tingkat severity dan priority. Tim harus membuat label sendiri, seperti P0-Critical, P1-High, P2-Medium, atau P3-Low. Jika aturan label tidak dibuat dengan jelas, bug yang penting dapat tercampur dengan bug ringan atau tugas pengembangan biasa.

c. Metrik QA bawaan masih terbatas

GitHub Issues dapat menampilkan issue yang terbuka, tertutup, atau diberi label tertentu. Namun, metrik QA yang lebih mendalam seperti defect leakage, defect density, reopened rate, dan bug aging belum tersedia secara otomatis. Untuk mendapatkan data tersebut, tim perlu mengolahnya secara manual atau menggunakan GitHub Projects dan tool tambahan.

d. Kualitas laporan bug bergantung pada kedisiplinan pengguna

GitHub Issues hanya menyediakan tempat untuk mencatat bug, tetapi kualitas isi laporan tetap bergantung pada reporter. Jika reporter tidak menuliskan langkah reproduksi, environment, hasil aktual, hasil yang diharapkan, dan bukti pendukung secara lengkap, developer akan kesulitan memahami serta memperbaiki bug.

e. Issue dapat menumpuk jika tidak ada triage rutin

Jika tim tidak melakukan triage secara berkala, issue yang masuk dapat menumpuk dan sulit dikendalikan. Bug yang valid, duplikat, tidak lengkap, atau sudah tidak relevan bisa bercampur dalam satu daftar. Kondisi ini dapat memperlambat proses perbaikan dan membuat prioritas kerja menjadi tidak jelas.

f. Diskusi dalam issue bisa terlalu panjang

GitHub Issues mendukung diskusi melalui komentar. Namun, jika komunikasi tidak dikelola dengan baik, satu issue dapat berisi terlalu banyak komentar. Hal ini dapat membuat informasi penting seperti keputusan perbaikan, hasil retest, atau penyebab bug menjadi sulit ditemukan kembali.

g. Kurang cocok untuk proyek non-teknis

GitHub Issues paling efektif digunakan pada proyek yang berhubungan dengan kode. Untuk proyek non-teknis atau organisasi yang tidak menggunakan GitHub sebagai repositori utama, fitur ini mungkin kurang optimal karena sebagian besar integrasinya dirancang untuk mendukung proses pengembangan perangkat lunak.

h. Membutuhkan pengaturan akses yang tepat

Pada repositori private atau proyek organisasi, pengaturan hak akses sangat penting. Jika permission tidak diatur dengan baik, informasi bug, kode, atau data sistem yang sensitif dapat diakses oleh pihak yang tidak seharusnya. Oleh karena itu, pengelolaan akses perlu dilakukan dengan hati-hati.

i. Bergantung pada ekosistem GitHub

GitHub Issues sangat bergantung pada platform GitHub. Jika tim menggunakan platform lain seperti GitLab, Bitbucket, atau sistem internal perusahaan, penggunaan GitHub Issues dapat membuat pekerjaan tersebar di beberapa tempat. Hal ini dapat mengurangi efisiensi dan menyulitkan proses pelacakan bug secara terpusat.

j. Kurang mendukung kebutuhan enterprise yang kompleks

Untuk perusahaan besar, GitHub Issues mungkin belum cukup jika dibutuhkan integrasi mendalam dengan sistem manajemen risiko, compliance, service desk, audit trail, atau laporan manajemen tingkat tinggi. Dalam kondisi tersebut, tool enterprise seperti Jira atau Azure DevOps mungkin lebih sesuai.

Tabel 5. Ringkasan Kelebihan dan Kekurangan GitHub Issues

Dimensi	Kelebihan	Kekurangan/Risiko
Kemudahan	Cepat dipelajari dan langsung tersedia di repository.	Butuh akun GitHub dan pemahaman dasar repository.
Integrasi kode	Issue dapat ditautkan ke branch, commit, dan pull request.	Kurang cocok jika proyek tidak berbasis kode.

Bug tracking	Label, assignee, milestone, komentar, dan attachment cukup untuk bug dasar.	Severity/priority perlu dibuat sendiri dengan label atau fields.
Pelaporan	Filter dan Projects membantu melihat progres.	Metrik QA yang detail tidak selengkap alat enterprise.
Kolaborasi	Diskusi, mention, dan lampiran membuat komunikasi jelas.	Diskusi bisa panjang jika tidak ada aturan triage.

9. Praktik Terbaik Implementasi GitHub Issues untuk Defect Management

GitHub Issues dapat digunakan secara efektif sebagai tool pelacakan bug, tim perlu menerapkan aturan kerja yang sederhana tetapi konsisten. Tujuannya bukan untuk membuat proses menjadi rumit, melainkan agar setiap bug yang dilaporkan dapat dipahami, diprioritaskan, diperbaiki, diuji ulang, dan ditutup dengan jelas. Dengan penerapan praktik terbaik, proses defect management menjadi lebih terstruktur dan mudah dievaluasi.

a. Gunakan template bug report

Template bug report sebaiknya digunakan agar laporan bug memiliki format yang seragam. Template minimal memuat deskripsi masalah, langkah reproduksi, hasil yang diharapkan, hasil aktual, environment, serta bukti pendukung seperti screenshot, video, atau log error.

b. Gunakan label standar

Label digunakan untuk mengelompokkan issue berdasarkan jenis, prioritas, atau area masalah. Contoh label yang dapat digunakan adalah bug, P0-critical, P1-high, P2-medium, P3-low, frontend, backend, payment, needs-info, duplicate, dan wontfix.

c. Pisahkan severity dan priority

Severity menjelaskan tingkat dampak bug terhadap sistem, sedangkan priority menjelaskan tingkat urgensi pengerjaannya. Keduanya perlu dibedakan karena bug dengan dampak besar belum tentu selalu dikerjakan paling awal apabila tidak memengaruhi rilis terdekat.

d. Tentukan assignee

bug yang sudah dinyatakan valid sebaiknya memiliki assignee atau penanggung jawab. Hal ini penting agar bug tidak dibiarkan menggantung dan tim mengetahui siapa yang bertugas melakukan perbaikan.

e. Gunakan milestone untuk rilis

Milestone dapat digunakan untuk mengelompokkan bug berdasarkan target rilis, sprint, atau versi aplikasi tertentu. Dengan milestone, tim dapat mengetahui bug mana saja yang harus diselesaikan sebelum rilis dilakukan.

f. Hubungkan issue dengan pull request

Issue sebaiknya dihubungkan dengan pull request yang berisi perbaikan kode. Penggunaan kata kunci seperti `fixes #nomor_issue` dapat membantu menautkan issue dengan pull request secara otomatis, sehingga hubungan antara laporan bug dan perbaikan kode menjadi jelas.

g. Lakukan triage secara rutin

Triage perlu dilakukan secara berkala untuk memeriksa issue baru, menentukan validitas bug, menghapus duplikasi, meminta informasi tambahan, serta menetapkan prioritas. Triage rutin membantu mencegah backlog menumpuk dan memastikan bug penting segera ditangani.

h. Tutup issue hanya setelah retest

Issue sebaiknya tidak langsung ditutup setelah developer mengirim perbaikan. QA atau tester perlu melakukan retest terlebih dahulu untuk memastikan bahwa bug benar-benar sudah tidak muncul kembali dan perbaikan tidak menimbulkan masalah baru.

i. Gunakan GitHub Projects sebagai board

GitHub Projects dapat digunakan untuk menampilkan bug dalam bentuk board atau tabel. Status seperti Backlog, To Do, In Progress, Review/QA, dan Done membantu tim memantau posisi setiap bug dalam alur kerja.

j. Evaluasi metrik secara berkala

Tim perlu mengevaluasi metrik defect secara berkala, seperti jumlah bug terbuka, bug prioritas tinggi, bug yang belum memiliki assignee, bug yang dibuka kembali, dan bug yang terlalu lama belum selesai. Evaluasi ini dapat digunakan untuk memperbaiki proses QA dan meningkatkan kualitas perangkat lunak.

10. Contoh Konfigurasi Defect Management

Berikut adalah contoh konfigurasi sederhana yang dapat digunakan jika sebuah tim memakai GitHub Issues untuk defect management. Konfigurasi ini dibuat mudah dipahami dan cocok untuk tugas kuliah.

Tabel 6. Contoh Label Severity dan Priority

Level	Severity - Dampak Masalah	Priority - Urgensi Pengerjaan
P0 / Critical	Aplikasi utama tidak bisa digunakan, data hilang, pembayaran gagal total, atau masalah keamanan serius.	Dikerjakan segera sebagai hotfix.
P1 / High	Fitur penting gagal dan banyak pengguna terdampak.	Masuk sprint berjalan atau patch terdekat.
P2 / Medium	Fungsi terganggu tetapi masih ada workaround.	Masuk backlog prioritas menengah.
P3 / Low	Masalah kecil seperti tampilan, teks, atau kasus jarang.	Dikerjakan jika ada waktu.
P4 / Trivial	Masalah kosmetik yang tidak mengganggu fungsi utama.	Dapat ditunda atau digabung dengan perbaikan lain.

Tabel 7. Contoh Field Bug Report yang Disarankan

Field	Isi yang Disarankan	Kegunaan
Title	Judul singkat dan spesifik.	Membuat bug mudah dicari.
Description	Penjelasan masalah secara ringkas.	Memberi konteks kepada developer.
Steps to reproduce	Langkah untuk mengulang bug.	Membantu developer memvalidasi bug.
Expected result	Perilaku yang seharusnya terjadi.	Menjadi standar hasil benar.
Actual result	Perilaku salah yang muncul.	Menjelaskan gejala bug.
Environment	Browser, OS, device, versi aplikasi, atau server.	Membantu mencari penyebab.
Evidence	Screenshot, video, log, atau error message.	Memperkuat laporan bug.

Label	bug, P1, frontend, backend, payment.	Memudahkan pengelompokan.
Assignee	Nama developer atau tim.	Menentukan penanggung jawab.

New issue: Bug report

Title: Checkout gagal setelah klik Bayar

Deskripsi masalah

Tuliskan bug secara singkat dan jelas.

Steps to reproduce

1. Buka halaman checkout 2. Pilih metode pembayaran 3. Klik Bayar

Expected result

Transaksi berhasil dan nomor pesanan muncul.

Actual result

Muncul pesan error 500 dan pesanan tidak tercatat.

Environment

Chrome 124, Android 13, staging

Evidence

Lampirkan screenshot/video/log error.

Metadata

• Label: bug, P1

• Assignee: @developer

• Milestone: Sprint 7

• Project: QA Board

• Type: Bug

Gambar 5. Contoh template issue untuk pelaporan bug

Tabel 8. Contoh Filter Pencarian GitHub Issues

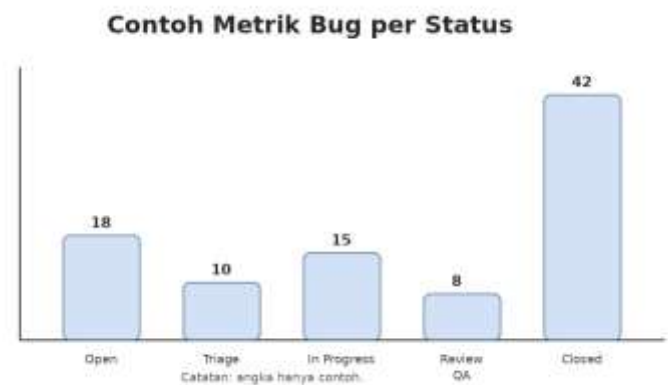
Tujuan	Contoh Filter	Keterangan
Bug terbuka	is:issue is:open label:bug	Menampilkan semua bug yang belum selesai.
Bug prioritas tinggi	is:issue is:open label:bug label:P1	Dipakai untuk triage harian.
Bug tanpa assignee	is:issue is:open label:bug no:assignee	Mencegah bug tidak memiliki pemilik.
Bug frontend	is:issue is:open label:bug label:frontend	Mencari masalah berdasarkan area teknis.
Bug dalam milestone	is:issue is:open label:bug milestone:"Sprint 7"	Memantau bug yang harus selesai dalam sprint.
Bug yang sudah ditutup	is:issue is:closed label:bug	Melihat riwayat bug selesai.

11. Contoh Metrik Defect yang Dapat Dipantau

Meskipun GitHub Issues sederhana, data issue tetap dapat digunakan untuk melihat kondisi kualitas software. Tim dapat memantau jumlah bug terbuka, bug berdasarkan label prioritas, bug yang belum memiliki assignee, bug dalam milestone tertentu, dan bug yang ditutup setelah pull request digabung. Untuk analisis yang lebih rapi, GitHub Projects dapat digunakan karena mendukung view, custom fields, automation, dan charts/insights (GitHub Docs, 2026e).

Tabel 9. Contoh Metrik Bug Tracking

Metrik	Tujuan	Contoh Interpretasi
Open bug	Melihat jumlah bug yang masih terbuka.	Jika jumlahnya terus naik, kapasitas perbaikan kurang.
Bug by priority	Melihat bug berdasarkan P0/P1/P2.	P0/P1 perlu ditangani lebih cepat.
Unassigned bug	Melihat bug yang belum punya penanggung jawab.	Bug bisa terlambat jika tidak ada assignee.
Bug aging	Mengukur lamanya bug belum selesai.	Bug lama menunjukkan bottleneck.
Reopened bug	Melihat bug yang dibuka kembali setelah dianggap selesai.	Bisa menandakan fix belum benar atau testing kurang lengkap.
Closed bug	Melihat bug yang sudah selesai.	Membantu mengevaluasi produktivitas dan kestabilan rilis.



Gambar 6. Contoh grafik metrik bug berdasarkan status

12. Referensi Foto, Screenshot, dan Video

Gambar yang dimasukkan pada laporan ini berupa ilustrasi/screenshot contoh yang dibuat berdasarkan fitur dan dokumentasi resmi GitHub. Link berikut dapat digunakan sebagai rujukan tambahan untuk foto, screenshot, dan video pembelajaran.

Tabel 10. Link Referensi Foto/Screenshot/Video

Jenis Referensi	Sumber	Link
Screenshot resmi fitur GitHub Issues	GitHub - GitHub Issues feature page	https://github.com/features/issues
Dokumentasi screenshot pembuatan issue	GitHub Docs - Creating an issue	https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/creating-an-issue
Dokumentasi screenshot filter issue	GitHub Docs - Filtering and searching issues	https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/filtering-and-searching-issues-and-pull-requests
Dokumentasi Projects	GitHub Docs - About Projects	https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects
Video tutorial penggunaan GitHub Issues	YouTube – GitHub Issues Tutorial	https://www.youtube.com/results?search_query=GitHub+Issues+tutorial
Tutorial latihan GitHub	GitHub Skills - Introduction to GitHub	https://github.com/skills/introduction-to-github

13. Kesimpulan

GitHub Issues merupakan tools pelacakan bug yang paling mudah dipilih untuk laporan ini karena sederhana, langsung menyatu dengan repository kode, dan cukup lengkap untuk kebutuhan defect management dasar. Dengan GitHub Issues, bug dapat dibuat sebagai issue, diberi label, ditugaskan kepada assignee, dimasukkan ke milestone, ditampilkan pada project board, dan ditautkan dengan pull request yang berisi perbaikan kode.

Dibandingkan Jira Software, GitHub Issues memang tidak memiliki workflow enterprise yang sangat detail. Dibandingkan Bugzilla, GitHub Issues terasa lebih modern dan familiar bagi pengembang yang sudah memakai GitHub. Oleh karena itu, GitHub Issues cocok digunakan oleh mahasiswa, tim kecil, proyek open source, dan tim pengembang yang ingin mengelola bug tanpa proses yang terlalu rumit.

Agar GitHub Issues efektif, tim tetap harus membuat aturan sederhana seperti template bug report, label prioritas, assignee yang jelas, milestone rilis, serta kebiasaan menautkan issue dengan pull request. Dengan cara tersebut, proses pelacakan bug dapat menjadi lebih rapi, transparan, dan mudah dievaluasi.

DAFTAR PUSTAKA

- GitHub. (2026). GitHub Issues: Project planning for developers.
<https://github.com/features/issues>
- GitHub Docs. (2026a). About issues. GitHub Docs. <https://docs.github.com/en/issues/tracking-your-work-with-issues/learning-about-issues/about-issues>
- GitHub Docs. (2026b). Creating an issue. GitHub Docs.
<https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/creating-an-issue>
- GitHub Docs. (2026c). Linking a pull request to an issue. GitHub Docs.
<https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/linking-a-pull-request-to-an-issue>
- GitHub Docs. (2026d). Filtering and searching issues and pull requests. GitHub Docs.
<https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/filtering-and-searching-issues-and-pull-requests>
- GitHub Docs. (2026e). About Projects. GitHub Docs.
<https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>
- GitHub Docs. (2026f). Using labels and milestones to track work. GitHub Docs.
<https://docs.github.com/en/issues/using-labels-and-milestones-to-track-work>
- GitHub Docs. (2026g). Managing issue types in an organization. GitHub Docs.
<https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/managing-issue-types-in-an-organization>
- GitHub Skills. (2026). Introduction to GitHub. GitHub. <https://github.com/skills/introduction-to-github>
- Nikeghbal, N., Kargaran, A. H., Heydarnoori, A., & Schütze, H. (2023). GIRT-Data: Sampling GitHub issue report templates. arXiv. <https://arxiv.org/abs/2303.09236>
- Proma, R. A., & Rosen, P. (2024). Visual analysis of GitHub Issues to gain insights. arXiv.
<https://arxiv.org/abs/2407.20900>